

NUBOM

Guía de Pruebas de la API REST

Endpoints, método HTTP y JSON sugerido para Postman / curl

Proyecto final · Módulo 7 · Diplomado en Desarrollo de Software

Base URL: <http://localhost:8080/api/v1>

Armando Luna

Septiembre 2026

Notas antes de probar

Base URL de todos los endpoints: `http://localhost:8080/api/v1`. No requieren sesión ni login (las rutas `/api/**` están excluidas del filtro de sesión de la app web). Los cuerpos van como JSON (Content-Type: `application/json`).

IDs de ejemplo ya sembrados por data.sql: hogar 1 = Casa Luna, 2 = Casa Rivera, 3 = Departamento Sur · inventario 1/2 = Refrigerador/Alacena de Casa Luna · tipo de inventario 1 = Refrigerador, 2 = Alacena · usuario 1 = Armando Luna (PROPIETARIO de Casa Luna), 2 = Usuario Uno (FAMILIAR de Casa Luna), 3 = Usuario Dos (sin hogar asignado, útil para probar la asociación N:M) · rol 1 = PROPIETARIO, 2 = FAMILIAR.

Sugerencia de orden para probar la relación 1:N inventario->producto: crear un hogar, crear un inventario para ese hogar, y agregar productos a ese inventario. Para la relación N:M usuario<->hogar: usar el usuario 3 (Usuario Dos, sin hogar) y asociarlo a un hogar existente.

1. Hogar — /api/v1/hogares

Método	Endpoint	Descripción	HTTP esperado
GET	/hogares	Lista todos los hogares.	200
GET	/hogares/{id}	Consulta un hogar por id.	200 · 404
POST	/hogares	Crea un hogar.	201 · 400
PUT	/hogares/{id}	Actualiza el nombre de un hogar.	200 · 404 · 400
DELETE	/hogares/{id}	Elimina un hogar (cascada 1:N: inventarios, productos, lista de compras).	204 · 404
GET	/hogares/{id}/inventarios	Inventarios del hogar (1:N).	200 · 404
POST	/hogares/{id}/inventarios	Crea un inventario para ese hogar (1:N).	201 · 404 · 400
GET	/hogares/{id}/lista-compra	Lista de compras del hogar (1:N).	200 · 404
POST	/hogares/{id}/lista-compra	Agrega un item a la lista de compras (1:N).	201 · 404 · 400
GET	/hogares/{id}/usuarios	Usuarios asociados a este hogar, con su rol (N:M, lado hogar).	200 · 404

POST /hogares

```
{
  "nombre": "Casa Rivera"
}
```

PUT /hogares/{id}

```
{
  "nombre": "Casa Rivera (editada)"
}
```

POST /hogares/{id}/inventarios

```
{
  "nombre": "Refrigerador",
  "capacidadMaxima": 20,
  "idTipo": 1
}
```

POST /hogares/{id}/lista-compra

```
{  
  "nombre": "Tortillas",  
  "categoria": "Panadería",  
  "cantidad": 1,  
  "unidad": "kg"  
}
```

2. Inventario — /api/v1/inventarios

Método	Endpoint	Descripción	HTTP esperado
GET	/inventarios[?idHogar=]	Lista inventarios; filtro opcional por hogar.	200 · 404 · 400
GET	/inventarios/{id}	Consulta un inventario por id.	200 · 404
PUT	/inventarios/{id}	Actualiza nombre/capacidad/tipo.	200 · 404 · 400 · 409
DELETE	/inventarios/{id}	Elimina un inventario (cascada 1:N: productos).	204 · 404
GET	/inventarios/{id}/productos	Productos del inventario (1:N).	200 · 404
POST	/inventarios/{id}/productos	Agrega un producto a ese inventario (1:N).	201 · 404 · 400 · 409

PUT /inventarios/{id}

```
{
  "nombre": "Refrigerador",
  "capacidadMaxima": 40,
  "idTipo": 1
}
```

POST /inventarios/{id}/productos

```
{
  "nombre": "Mermelada",
  "cantidad": 1,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

El 409 en PUT ocurre si la nueva capacidad es menor a los productos ya guardados, o si no es 20/40/80. El 409 en POST ocurre si el inventario ya alcanzó su capacidad máxima.

3. Producto — /api/v1/productos

Método	Endpoint	Descripción	HTTP esperado
GET	/productos[?idHogar=][&idInventario;=]	Lista productos; filtros opcionales por hogar y/o inventario.	200 · 404 · 400
POST	/productos	Crea un producto indicando el inventario en el cuerpo.	201 · 404 · 400 · 409
GET	/productos/{id}	Consulta un producto por id.	200 · 404
PUT	/productos/{id}	Actualiza un producto.	200 · 404 · 400
DELETE	/productos/{id}	Elimina un producto.	204 · 404

POST /productos

```
{
  "idInventario": 1,
  "nombre": "Mermelada",
  "cantidad": 1,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

PUT /productos/{id}

```
{
  "nombre": "Mermelada de fresa",
  "cantidad": 2,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

4. Lista de compras — /api/v1/lista-compra

Método	Endpoint	Descripción	HTTP esperado
GET	/lista-compra[?idHogar=]	Lista todos los items (de todos los hogares); filtro opcional por hogar.	200 · 404 · 400
GET	/lista-compra/{id}	Consulta un item por id.	200 · 404
PUT	/lista-compra/{id}	Actualiza un item (incluye marcar comprado).	200 · 404 · 400
DELETE	/lista-compra/{id}	Elimina un item.	204 · 404

PUT /lista-compra/{id}

```
{
  "nombre": "Tortillas",
  "categoria": "Panadería",
  "cantidad": 2,
  "unidad": "kg",
  "comprado": true
}
```

5. Tipos de inventario — /api/v1/tipos-inventario

Método	Endpoint	Descripción	HTTP esperado
GET	/tipos-inventario	Lista el catálogo de tipos.	200
GET	/tipos-inventario/{id}	Consulta un tipo por id.	200 · 404
POST	/tipos-inventario	Crea un tipo de inventario.	201 · 400
PUT	/tipos-inventario/{id}	Actualiza un tipo de inventario.	200 · 404 · 400
DELETE	/tipos-inventario/{id}	Elimina un tipo (rechaza si está en uso).	204 · 404 · 409

POST /tipos-inventario

```
{
  "nombre": "Congelador",
  "descripcion": "Almacenamiento a muy baja temperatura",
  "icono": "bi-snow"
}
```

PUT /tipos-inventario/{id}

```
{
  "nombre": "Refrigerador",
  "descripcion": "Almacenamiento refrigerado para perecederos",
  "icono": "bi-snow2"
}
```

Prueba el 409: intenta borrar el tipo 1 (Refrigerador) o 2 (Alacena) — ambos están en uso por los inventarios sembrados en data.sql.

6. Usuario — /api/v1/usuarios

Incluye el CRUD del usuario y el lado “usuario” de la relación N:M usuario<->hogar (tabla intermedia usuario_hogar, con el rol como atributo de la asociación).

Método	Endpoint	Descripción	HTTP esperado
GET	/usuarios	Lista todos los usuarios (sin exponer la contraseña).	200
GET	/usuarios/{id}	Consulta un usuario por id.	200 · 404
POST	/usuarios	Crea un usuario.	201 · 400 · 409
PUT	/usuarios/{id}	Actualiza nombre/correo/activo.	200 · 404 · 400 · 409
DELETE	/usuarios/{id}	Elimina un usuario (rechaza si tiene membresías, ver nota).	204 · 404 · 409
GET	/usuarios/{idUserio}/hogares	Hogares asociados a este usuario, con su rol (N:M, lado usuario).	200 · 404
POST	/usuarios/{idUserio}/hogares/{idHogar}	Asocia (N:M) al usuario con el hogar, con el rol indicado.	201 · 404 · 400 · 409
DELETE	/usuarios/{idUserio}/hogares/{idHogar}	Elimina la asociación N:M.	204 · 404

POST /usuarios

```
{
  "nombre": "Ana Torres",
  "correo": "ana.torres@correo.com",
  "contrasena": "clave123"
}
```

PUT /usuarios/{id}

```
{
  "nombre": "Ana Torres López",
  "correo": "ana.torres@correo.com",
  "activo": true
}
```

POST /usuarios/{idUserio}/hogares/{idHogar}

```
{
  "idRol": 2
}
```

Flujo sugerido para el N:M: POST /usuarios/3/hogares/2 (Usuario Dos -> Casa Rivera, idRol=2) -> GET /usuarios/3/hogares y GET /hogares/2/usuarios (deben coincidir) -> POST de nuevo con el mismo par (409, ya asociado) -> DELETE /usuarios/3/hogares/2 (204) -> GET /usuarios/3/hogares (vacío otra vez). El 409 en DELETE /usuarios/{id} aparece si el usuario todavía tiene una membresía activa: elimina primero la asociación.

7. Rol — /api/v1/roles

Catálogo de roles (PROPIETARIO, FAMILIAR): el atributo propio de la relación N:M usuario<->hogar.

Método	Endpoint	Descripción	HTTP esperado
GET	/roles	Lista todos los roles.	200
GET	/roles/{id}	Consulta un rol por id.	200 · 404
POST	/roles	Crea un rol.	201 · 400
PUT	/roles/{id}	Actualiza un rol.	200 · 404 · 400
DELETE	/roles/{id}	Elimina un rol (rechaza si tiene membresías activas).	204 · 404 · 409

POST /roles

```
{
  "nombre": "INVITADO",
  "descripcion": "Acceso de solo lectura al hogar"
}
```

PUT /roles/{id}

```
{
  "nombre": "FAMILIAR",
  "descripcion": "Puede consultar y actualizar el inventario del hogar"
}
```

Prueba el 409: intenta borrar el rol 1 (PROPIETARIO) o 2 (FAMILIAR) — ambos tienen membresías activas sembradas en data.sql.